Mittuniversitetet
MID SWEDEN UNIVERSITY

Enhancing Code Static Analysis with LLMs: An Experiment with SpotBugs and Gemini

Johan Svärd and Morgan Teräs

Final Project
Main field of study: Computer Engineering BA (C)
Credits: 15
Semester/year: Spring 2025
Supervisor: Truong Ho-Quang
Examiner: Raja-Khurram Shahzad
Course code/registration number: DT133G
Programme: Software Engineering

Enhancing Code Static Analysis with LLMs: An Experiment with SpotBugs and Gemini

Johan Svärd and Morgan Teräs
Department of Communication, Quality Management and Information Systems
Mid Sweden University
Östersund, Sweden
{josv2204, mote2200}@student.miun.se

***Abstract*—Context: Static analysis tools are frequently employed in software projects to identify issues and flaws in the code. However, these tools frequently produce many false positives, i.e., bug reports that are not actual code flaws, that require developers' time and effort to adjudicate. Objective: We aim to integrate a Large Language Model (LLM), Gemini, with a static analysis tool, SpotBugs, in our tool, LLMAS, to automatically filter false positives from SpotBugs' reports. Method: We test LLMAS on 10,056 test cases from the Juliet Java 1.3 dataset, covering 14 CWEs, and then compare the results to SpotBugs in terms of precision, recall rate, and processing time. Results: The results for LLMAS show a precision of 0.3673 and a recall rate of 0.4914 compared to SpotBugs' precision of 0.3236 and the same recall rate. The additional time taken for LLMAS compared to SpotBugs is very large, as our experiment shows that standalone SpotBugs took 7.2 minutes while LLMAS took 4.86 hours. Conclusion: We conclude from our experiment that even though LLMAS performs better than SpotBugs, the improvement most likely does not justify the additional time taken in most software projects. However, we see potential in this methodology and urge other researchers to explore similarly straightforward approaches, such as LLMAS, with optimized prompts and LLM parameters.**

***Index Terms*—Static code analysis, Large language models, False positive filtering**

Acknowledgment

We want to express our sincere thanks to our supervisor, Truong Ho-Quang, for his guidance and support throughout this thesis. We are also very grateful to Sergio Rico, the course responsible.

I. Introduction

Code quality is a multifaceted concept that varies depending on the context and perspective. It generally encompasses characteristics that make code efficient, readable, and usable. High-quality code is not only free of defects but also designed to be understandable, modifiable, portable, and reusable. These attributes are essential for the software's immediate functionality, long-term maintainability, and adaptability [1].

One of the primary techniques to assess and improve code quality is static analysis. In contrast to dynamic analysis, which involves evaluating the program through execution, static analysis is the practice of evaluating the source code without executing it. Static analysis can be performed either manually, through practices such as code review or program comprehension, or automatically through specialized tools. In most cases, static analysis refers to automated analysis performed by such tools [2].

Recent advancements in artificial intelligence (AI), particularly large language models (LLMs), have significantly influenced the field of static analysis. LLMs are AI systems trained on large collections of natural language and code, allowing them to perform tasks such as code completion, bug detection, and natural language understanding [3]. Several studies have explored the integration of LLMs with existing static analysis tools to enhance their capabilities (e.g., Li et al. [4]; Li et al. [5]). Other research efforts have focused on employing LLM-based systems as standalone tools capable of detecting code issues independently (e.g., Fan et al. [6]; Ma et al. [7]). Across these studies, there is a common consensus on the promising avenue of using LLMs to improve the performance of conventional static analysis tools, thereby improving overall code quality.

Further research by Wang et al. [8] provides a comprehensive literature review on the application of LLMs in program analysis. The reviewed studies are categorized into static, dynamic, and hybrid analysis approaches, each approach is further subdivided to illustrate specific use cases of LLMs within those frameworks. The authors argue that integrating LLMs into static analysis workflows can help address limitations such as high false positive rates, performance overhead, and the trade-off between speed and accuracy. However, they also highlight several challenges, including model hallucinations, prompting effectively, token limitations, and high cost.

Although research on utilizing LLMs with static analysis tools has yielded several papers, further research is still necessary. More specifically, to the best of our knowledge, a gap exists in utilizing LLMs to filter false positives from traditional static analysis tools. Filtering false positives alleviates developers from spending a significant amount of time and effort manually adjudicating all the warnings from static analysis tools. This research direction has been explored by Li et al. [4], Kharkar et al. [9], and Wen et al. [10], but expanding upon these works by evaluating other implementations utilizing different static analysis tools and LLMs on a broader range of issues is still needed, and mentioned by them as suggested future work. Therefore, our aim with this thesis is to present and benchmark our tool, LLMAS, that utilizes Google Gemini to filter out false positives from SpotBugs reports. LLMAS

will be evaluated by examining 14 CWEs with test cases from the Juliet Java 1.3 dataset[1].

The outline for this thesis is as follows. In *Purpose and Contributions*, we present the problem we aim to solve, our research questions, and the expected contributions of this thesis. In *Background*, related concepts and terminologies to this thesis are presented and explained. *Related work* consists of papers related to this thesis; we also present how these relate to our work and how they differ. In *Research Methodology*, we present and rationalize how we performed our research regarding implementation details of our tool and benchmarking. *Results* consists of the answers to our research questions posed in *Purpose and Contributions* that were obtained from our experiment. In *Discussion*, we reason and discuss the results and their implications. Finally, *Conclusions* consists of a summary of the thesis and the findings.

II. Purpose and Contributions

Static analysis tools are often utilized in the software development process. Still, their results can yield large numbers of false positives that require considerable time and effort for developers to adjudicate. Therefore, to the best of our knowledge, further research is needed focusing on how LLMs can be integrated with existing static analysis tools to improve this process. Specifically, how LLMs can be used to evaluate the results of static analysis tools to filter out false positives. Lavazza et al. [11] highlights this sentiment in their paper, stating that "Among the future possible evolutions of this work, the most intriguing one concerns studying the possibility of replacing inspection via some sort of AI-based models that can discriminate false positives and true problems." Suggesting that LLMs could enhance or completely replace manual inspections required after using static analysis tools.

Li et al. [4], Kharkar et al. [9], and Wen et al. [10] have all developed tools that integrate LLMs with static analysis tools to filter out false positives with promising results, as suggested by Lavazza et al. [11]. The success of this approach would benefit both developers and stakeholders by minimizing the time and resources spent on manual code reviews and increasing overall software reliability. LLM-assisted static analysis tools can leverage machine learning models to better distinguish between true and false positives than traditional static analysis tools, thereby reducing alert fatigue among developers. We aim to build upon their work by integrating LLMs to filter false positives of static analysis tools, benchmarking our tool on 14 CWEs with test cases from the Juliet Java 1.3 dataset[1], none of which have been covered by these works. We will also present an alternative implementation.

This thesis explores how an LLM, Google Gemini, can be integrated with an existing static analysis tool, SpotBugs, to create an LLM-assisted static analysis tool to improve code quality assessment. To this extent, we present LLMAS (**L**arge **L**anguage **M**odel **A**ssisted **S**tatic Analysis). Specifically, this thesis investigates whether LLMAS can enhance traditional static analysis by reducing false positives and improving precision regarding several code quality issues.

To achieve this, we define the following research questions:

RQ1 *How does LLMAS compare to traditional static analysis tools regarding precision and recall rate?*

RQ2 *How much additional processing latency does using LLMs for static code analysis introduce?*

The contributions of this project include:

- A comparative analysis of LLMAS versus traditional methods, including empirical data on precision and recall rate.
- A prototype implementation of an LLM-assisted static analysis tool, LLMAS, that integrates Google Gemini with SpotBugs.

III. Background

This section will explain the concepts and terminologies used throughout the thesis.

A. Code Quality and Security

The term code quality is extensive. It can refer to the code being bug-free or efficient in terms of functionality and readability. Although generally, code quality relates to how developer-friendly the code is. High-quality code is of great importance in software projects, as it reduces the time and effort required for developers to read, understand, and extend the code, both their own and that written by other team members. Another benefit is that it leads to more secure software since it forces developers to adhere to best coding practices that enforce security and reduce flaws and vulnerabilities [1].

Producing secure software is not only desirable but a necessity. Security in software refers to protecting the software's information and resources from unauthorized users while ensuring that the software is resistant to attacks and malicious intent from malicious actors. The goal is to prevent, detect, and recover from these attacks. Failure to remove vulnerabilities in the code can lead to cyberattacks, data breaches, and significant disruptions to the system. Furthermore, these attacks can lead to data loss and unavailability, potentially compromising the user's confidentiality and integrity [12].

When discussing vulnerabilities, flaws, or weaknesses in code, it is highly beneficial to share a common terminology. Therefore, Common Weakness Enumeration (CWE) was created. CWE is a community-developed list containing software and hardware weaknesses that could lead to vulnerabilities. These weaknesses are referred to as CWEs. CWEs are widely used in software development and research to discuss the mitigation and elimination of potential weaknesses. They are also used in software analysis tools to evaluate the tool's effectiveness against different types of flaws and serve as a standard for identification.

A CWE consists of an identification number for the weakness, along with a short descriptive name, a longer description, the impacts of the weakness, and examples of the weakness in

[1] https://samate.nist.gov/SARD/test-suites/111

code, among other elements [13]. For example, SQL Injection[2] is referenced as CWE-89, where 89 is the identification number. The full name is 'Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')'. HTTP Request/Response Splitting is referred to as 'CWE-113: Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Request/Response Splitting')'[3], and XPath Injection is referred to as 'CWE-643: Improper Neutralization of Data within XPath Expressions ('XPath Injection')'[4].

B. Static Analysis Tools

Program analysis is often used in software development to determine code and software quality. Program analysis involves examining software behavior in terms of correctness, safety, and performance. There are two methods of conducting these analyses: static analysis, in which the code is analyzed without execution, and dynamic analysis, in which the code is executed, such as through unit testing. A hybrid approach of the two can also be performed. These methods often use specialized tools to perform analysis, which makes the process more effective [14].

Static analysis tools are frequently employed in software development to evaluate the quality of code. This analysis can be performed on the source code or by examining the compiled code. These tools can be integrated into the workflow using CI/CD tools, such as GitHub Actions[5], by incorporating them into the IDE, or using them standalone on any given code. All methods serve the same purpose of alerting the developer to potential issues in the code, such as incorrect syntax, unused variables, or more severe issues, including vulnerabilities to SQL injection[2] and HTTP response splitting[3]. Although static analysis tools can often suffer from high amounts of false positives, false negatives, and inability to detect runtime issues, such as invalid input, they are still utilized for their ease of use, scalability, and ability to detect less complex problems, such as naming conventions, with high confidence [15].

SpotBugs is an open-source static analysis tool available on GitHub[6]. SpotBugs analyzes Java projects and files through the compiled code, i.e., JAR or class files. It can be used as a standalone tool through a GUI or the CLI, but it can also be integrated into build tools such as Ant[7], Maven[8], and Gradle[9] or the IDE Eclipse[10]. SpotBugs can detect about 400 bug patterns through its analysis, and can also be extended with plugins to detect a wider variety of bugs, for example, Find Security Bugs[11] that extend SpotBugs to detect 141 CWEs related to security. Furthermore, the analysis process can be modified to only specific classes, bug types, severity levels, and more [16].

[2]https://cwe.mitre.org/data/definitions/89.htm
[3]https://cwe.mitre.org/data/definitions/113.html
[4]https://cwe.mitre.org/data/definitions/643.html
[5]https://github.com/features/actions
[6]https://github.com/spotbugs/spotbugs
[7]https://ant.apache.org/
[8]https://maven.apache.org/
[9]https://gradle.org/
[10]https://eclipseide.org/
[11]https://find-sec-bugs.github.io/

C. Artificial intelligence and Large language models

Artificial intelligence has rapidly gained popularity in recent years and is now a household term, despite ongoing research dating back much further than this. Artificial intelligence is a vast term, and what is often referred to as artificial intelligence are the generative models that are trained on very large amounts of data and are thus able to generate content such as text, images, and videos [17]. One of the most popular uses of generative artificial intelligence is large language models (LLMs), which are trained to understand and generate text.

LLMs first gained significant traction with the introduction of OpenAI's Chat-GPT models[12], and since then, many more models have emerged, such as Meta's Llama[13], Anthropic's Claude[14], and Google's Gemini[15]. These models have a great understanding of text and natural language. They can closely mirror the abilities of a human in this regard while also being highly proficient in text generation. Along with use cases such as writing, summarizing, and translating, they can also be applied in software development contexts, as they can understand, generate, and analyze code. Therefore, developers often use them to assist them in tasks such as debugging, generating boiler-plate code, and gathering information about using a language or code library [18].

The LLM Gemini is developed by Google and has its start in 2013 with Google's research on generative artificial intelligence. Gemini can perform the tasks previously mentioned, such as high-level understanding and text generation, and offers integration with other Google services, including Gmail[16], Google Docs[17], and more. Gemini can be accessed through its dedicated website, iPhone or Android application, or by its API. Several models of Gemini are available and differ in their context windows, token costs, and general performance in terms of understanding and generation [19].

D. Evaluation Metrics

Evaluation metrics are used to assess a model or tool based on true positives, false positives, true negatives, and false negatives. These terms refer to the classifications of the tool or model; for example, in a static analysis tool, true positives are the number of actual bugs that are correctly identified as such by the tool. True negatives are all the non-bugs that were not marked as bugs, i.e., the code marked as issue-free and is truly issue-free. False positives refer to all issue-free code that the tool reports as a bug. False negatives are all actual bugs that are not reported by the static analysis tool. Therefore, false positives and false negatives refer to misclassifications by the tool, while true positives and true negatives refer to the correct classifications. [20]

[12]https://openai.com/chatgpt/overview/
[13]https://www.llama.com/
[14]https://www.anthropic.com/claude
[15]https://deepmind.google/technologies/gemini/
[16]https://workspace.google.com/intl/sv/gmail/
[17]https://workspace.google.com/products/docs/

Several types of metrics can be calculated from these classifications, such as precision and recall rate. Precision is the proportion of the classified positives that are truly positive. For static analysis tools, this means the fraction of reported bugs that are actual bugs. Recall rate refers to the proportion of all the positives correctly marked as such. For a static analysis tool, this would mean the fraction of total bugs correctly reported as bugs. The relevance of the metric can depend on the object being evaluated, its goal, and the dataset used. [21]

IV. Related Work

Li et al. [4] present LLift, an LLM-assisted static analysis tool that integrates with UBITect to detect use-before-initialization (UBI) bugs in the C language. The motivation for this tool stems from the challenges currently facing static analysis tools, which include the trade-off between too many false positives and missing bugs. The experiment detects UBI bugs in the Linux kernel, where an LLM filters the results from UBITect to reduce the false-positive rate. The results show a precision of 50% and a recall rate of 100%. Note that the recall rate here is compared to the UBITect and not from all issues in the Linux kernel. For future work, they suggest looking at other types of bugs to investigate the general performance of this approach, which aligns with the aim of our work.

Kharkar et al. [9] discuss the current problems with static analysis tools and mention the high false-positive rate as a significant reason why these techniques may go unused in software projects. They propose integrating with LLMs to filter false positives from the tool Infer. Their experiments examine Java, C++, and C# code to detect null dereference issues. The dataset comprises a curated set of issues detected by Infer and subsequently manually labeled as true or false positives. The benchmark results show an increase in precision but a lower recall rate. Their experiments confirm this by detecting resource leaks, which exhibit a similar pattern of increased precision but decreased recall rate. They suggest using this approach to false positive filtering for other issues as a direction for future work, aligning with our goals in this thesis.

Wen et al. [10] present the tool LLM4SA, which integrates LLMs with the static analysis tools Cppcheck, CSA, and Infer to inspect bug warnings. LLM4SA utilizes LLMs to examine bug reports and related code snippets, assigning a confidence rating to each bug on a scale of 0 to 1. CodeQuery retrieves the relevant code, which refers to all the code mentioned in the report and the code that uses that code. Benchmarking was performed on the Juliet test suite by NIST and on 11 real software projects to detect six types of bug occurrences in C and C++ code: null pointer dereference, uninitialized variable, use-after-free, divide by zero, memory leak, and buffer overflow. In the experiments, Chat-GPT and LLama were the LLMs used, and the results show a precision of 81.13%, a recall of 96.64%, and an accuracy of 79.5%, but these numbers differ depending on the bug. Our work is similar to this one, as we will utilize the Juliet dataset, but our tool will examine Java code instead of C and C++ code. Additionally, our benchmarks will not include the same set of bugs.

Author	Langs	SA	LLMs	Data	CWEs
Li et al. LLift	C	UBITect	GPT-4	Linux Kernel	457, 908
Kharkar et al.	Java, C++, C#	Infer	Code-BERTa, GPT-C	7 real world projects	404, 476
Wen et al. LLM4SA	C, C++	Cpp-check, CSA, Infer	Chat-GPT, LLaMA	Juliet C/C++ 1.3, 11 real world projects	121, 122, 124–127, 369, 401, 415, 416, 457, 476, 690, 824
LLMAS	Java	SpotBugs	Gemini Flash Lite	Juliet Java 1.3	15, 78, 89, 90, 113, 259, 319, 321, 327, 329, 539, 601, 614, 643

TABLE I
Comparison of similar studies in Related Works

Kaur and Nayyar [22] compare different static analysis tools in their paper. The tools tested include both C/C++ and Java tools. The Java tools, SpotBugs with Find Security Bugs and PMD, are tested on the open-source software Apache Tomcat. They found that SpotBugs performs better than PMD, detecting 92 out of 171 fault types compared to the 46 detected by PMD. The C/C++ tools tested are Flawfinder, RATS, and CPPCheck. These tools are tested on the Juliet C/C++ 1.3 dataset. This work relates to ours because it uses SpotBugs with the Find Security Bugs plugin. They also show the competency of this tool, validating our choice of this in LLMAS. Additionally, they also use the Juliet dataset. However, while they use the C/C++ version, we use the Java version. To determine the tool's effectiveness, they utilize the manifest file available in Juliet, which differs from how we determine the tool's precision and recall rate, as we obtain the ground truth from a script. We found that this approach yields more accurate results.

Flynn and Klieber [23] present their findings of researching false positive filtering with a specific perspective of the Department of Defense. They propose an approach to adjudicate false positives using static analysis tools and LLMs; this approach utilizes some script automation, but otherwise, no artifact is presented. Their approach involves formatting the prompts to the LLM in a way that requests more code if needed for the analysis, thereby reducing token consumption. They also make the LLM double-check its work to find any flaws it might have made. This step may seem redundant, but examples of the LLM correcting its own mistakes are provided. They do not showcase any benchmarking statistics, but highlight real-world examples of the approach in action, and all the API chat history is publicly available. They suggest expanding

their work to include more types of flaws and more static analysis tools. This paper aligns with our aim to reduce false positives using LLMs. Still, our work differs in that we will focus mainly on the artifact we develop, rather than adopting a general approach to accomplish this. Additionally, we will act upon their future work suggestions by examining different types of flaws and utilizing other static analysis tools.

Li et al. [5] present a static analysis tool, IRIS, which can perform analysis of whole code repositories. In IRIS, CodeQL is combined with LLMs to detect security vulnerabilities in Java code. The dataset used for the benchmarks consists of manually curated vulnerabilities. The results show that IRIS identifies approximately double the number of issues compared to CodeQL, and that the false-positive rate decreased by 5%. It also contains a component to filter out false positives. Although no direct comparison will be made to our work, it will guide and inspire our implementation.

Fan et al. [6] have done similar research in their paper. They present ICAA, an approach to utilize LLMs in static analysis. This approach combines LLMs with traditional techniques, which could include static analysis tools, but is not a significant component. Two implemented agents are showcased, focusing on analyzing bug detection and code intention. This approach comprises a false pruner component, which is closely related to the work we aim to accomplish. The benchmark results show that the false-positive rate decreased from 85% in the baseline to 66% with a recall rate of 60.8%. The data comprises the "Non-functional bugs dataset" and a few curated issues. One weakness of the benchmark is that the baseline used is a standalone LLM analysis, which could lead to more favorable results for ICAA. Challenges include a relatively high token consumption, with usage ranging from 355 to 996 tokens per line of code. However, the performance aligns closely with available state-of-the-art techniques, implying further research in this area is of great interest.

Both the work of Fan et al. [6] and Li et al. [5] regard the use of LLMs as static analysis tools. What makes our research unique in comparison is that we aim to combine an already available tool with an LLM to specifically filter out false positives. Unlike these papers that produced LLM-driven tools, we will present an LLM-assisted tool and aim to benchmark the strategy of using an LLM to filter out false positives.

Ma et al. [7] perform a more general experiment investigating code comprehension of LLMs. This work is done by providing the LLM with several tasks to solve. These tasks are divided into syntax, dynamic, and static comprehension. Four models and four languages are utilized in the experiments, and the results generally show a pretty good understanding of syntax and static behavior, while lacking in dynamic understanding. The primary concern raised in this paper is the hallucination inherent in these models.

V. Research Methodology

This section outlines the research approach for the experiment, including the hypothesis testing that guides the evaluation, the rationale for tool selection, the construction of LLMAS, the data collection process, and the definition of the evaluation metrics used.

A. Hypothesis

To evaluate the effectiveness of LLMAS compared to the standard version of SpotBugs, the following hypotheses are formulated from each of the evaluation metrics examined in the experiment:

- **Precision**
  - $\mathbf{H_0}$: *There is no difference in precision between LLMAS and SpotBugs.*
  - $\mathbf{H_A}$: *There is a difference in precision between LLMAS and SpotBugs.*
- **Recall Rate**
  - $\mathbf{H_0}$: *There is no difference in recall rate between LLMAS and SpotBugs.*
  - $\mathbf{H_A}$: *There is a difference in recall rate between LLMAS and SpotBugs.*
- **Processing Time**
  - $\mathbf{H_0}$: *There is no difference in processing time between LLMAS and SpotBugs.*
  - $\mathbf{H_A}$: *There is a difference in processing time between LLMAS and SpotBugs.*

B. Static Code Analysis Tool Selection

We selected SpotBugs version 4.9.3[18] as this study's static code analysis tool. Our decision was based on the following criteria: support for the Java programming language, open-source availability, demonstrated effectiveness in detecting software issues, and a strong track record of use and active maintenance. SpotBugs met all of these requirements. It is specifically designed for analyzing Java code, and being open-source enables other researchers to replicate our analysis and validate the results.

SpotBugs' effectiveness and reliability have been demonstrated in prior research [5], [11], [22], and it is widely adopted in both academic and industrial contexts. To expand the tool's capabilities in identifying security-related issues, we included the Find Security Bugs plugin[19]. This extension enhances SpotBugs by adding a comprehensive set of rules specifically tailored to detect common security vulnerabilities in Java applications, thereby broadening the scope of our static analysis. Other static analysis tools were considered but ultimately did not meet our criteria as effectively.

C. Generative AI Model Selection

For this thesis, we selected Google Gemini Flash Lite version 2.0[20] as the LLM. Our decision was based on the following criteria: well-documented API, low usage costs, compatibility with static code analysis workflows, and effectiveness in language understanding for source code contexts.

[18] https://github.com/spotbugs/spotbugs/releases/tag/4.9.3
[19] https://github.com/find-sec-bugs/find-sec-bugs
[20] https://deepmind.google/technologies/gemini/flash-lite/

Gemini Flash Lite met all of these requirements. It offers a lightweight yet capable model suitable for high-volume inference tasks, with low token pricing that makes large-scale analysis feasible. Alternative LLMs such as OpenAI's GPT-4[21] were considered but excluded primarily due to higher usage costs and limited access to lighter-weight model variants. Gemini Flash Lite provided the most balanced trade-off between performance and cost.

D. LLMAS Construction

A prototype was developed to integrate LLM analysis from Google Gemini with SpotBugs; LLMAS (**L**arge **L**anguage **M**odel **A**ssisted **S**tatic Analysis)[22]. This integration aims to enhance the static analysis output by reducing the number of false positives. LLMAS is designed around a multi-component pipeline that facilitates automated static analysis and LLM-based refinement of bug reports.

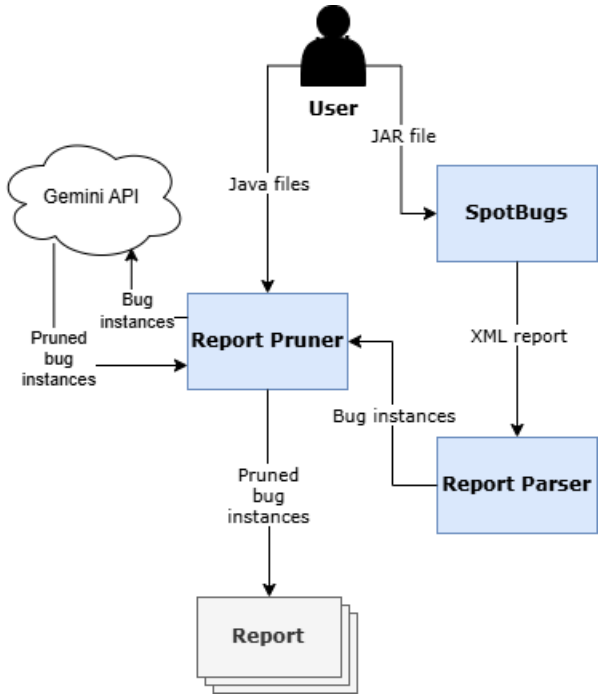


Fig. 1. Tool Data Flow

As illustrated in Figure 1, LLMAS operates through several key steps. Initially, the user uploads a JAR file to be analyzed. At this time, the user also provides Java files associated with this JAR file. The analysis is only run on the Java files provided, so it is possible to only provide a subset of files from the project if that is desirable. Comments within the Java files are removed before analysis to limit adverse side effects produced by Gemini.

When the analysis is started, SpotBugs analyzes the JAR file and produces an XML report of all the bugs found. SpotBugs is executed with the relaxed setting and high effort to report more bugs in the code at the cost of more of them being false positives. Still, this is desirable, as we aim for high precision and intend to prune these. The XML report is then processed by the Report Parser to extract each bug as an internal data type, represented as a class with variables for type, priority, rank, category, CWE-ID, source file path, explanation, start line, and end line.

The Report Pruner functions as the central integration component of the system. It receives the bug instances from the Report Parser, Java source code from the user, and feedback from the Gemini API. The pruner generates prompts to Gemini for each Java source file with the bug instances and source code. The prompts are formatted as:

> *For the given code and bugs, filter out all the bugs that are false positives. Respond with a subset of the same bugs, with the same values, that are true positives present in the code. If no bugs are true positives, respond with nothing. IMPORTANT: Only respond with bugs present in the provided bugs list below, NO BUG RESPONSES ALLOWED IF NOT PRESENT THERE!*
> *Code:*
> ...
> *Bugs:*
> ...

During testing, we found that the tool could sometimes respond with bugs not reported by SpotBugs. Therefore, the instruction to respond only with bugs present in the request is necessary. The file's source code is provided under "Code:", and the bug instances are provided under "Bugs:" in the JSON format. This prompt is then sent to Gemini through the API, which analyzes the provided code and bugs and responds with a pruned list of bug instances.

To ensure the response is a list of bug instances, structured output is utilized. Structured output is a technique supported by the Gemini API that allows a data type to be specified along with the prompt, enabling responses to be received only in that format. In this case, a list containing bug instances was specified. The temperature for Gemini is set to 0 to ensure deterministic results, as described in the Gemini documentation [24]. Other than temperature, the default configurations for Gemini were used.

Using Gemini's analysis, the Report Pruner refines the initial bug list by removing false positives and less pertinent findings. Finally, the refined bug instances and the non-pruned bug instances are consolidated into comprehensive JSON reports and displayed to the user as graphs in the GUI.

By integrating SpotBugs with Gemini, LLMAS aims to significantly reduce developer overhead in assessing false static analysis results, thereby improving overall productivity and code quality. The feedback from Gemini helps evaluate the relevance and accuracy of each bug instance. This design enables a hybrid approach that combines traditional static analysis with advanced language model understanding. The

[21] https://openai.com/gpt-4
[22] https://bitbucket.org/jsvard/thesis-tool/src/main/

prototype's User Interface (UI) features simple interfaces for uploading JAR and source files, as well as viewing raw and pruned reports.

E. Data Collection

To evaluate LLMAS, the Juliet Java 1.3 dataset[23] from the NIST Software Assurance Reference Datasets (SARD) [25] was used. Developed by the NSA Center for Assured Software, this dataset contains Java code samples with known quality issues and comprises 28,881 manually crafted test cases, covering 112 different CWEs, totaling approximately 73.2 MB in size. It is widely recognized within the research community for benchmarking software assurance tools [26].

Of the 112 CWEs in Juliet, we evaluated LLMAS on the test cases relating to 14 of these, which contain 10,056 test cases. The CWEs examined are:

- **CWE-15:** External Control of System or Configuration Setting
- **CWE-78:** Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
- **CWE-90:** Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')
- **CWE-113:** Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Response Splitting'
- **CWE-259:** Use of Hard-coded Password
- **CWE-319:** Cleartext Transmission of Sensitive Information
- **CWE-321:** Use of Hard-coded Cryptographic Key
- **CWE-327:** Use of a Broken or Risky Cryptographic Algorithm
- **CWE-329:** Not Using a Random IV with CBC Mode
- **CWE-539:** Information Exposure Through Persistent Cookies
- **CWE-601:** URL Redirection to Untrusted Site ('Open Redirect')
- **CWE-614:** Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
- **CWE-643:** Improper Neutralization of Data within XPath Expressions ('XPath Injection')

These 14 were selected for their support by Find Security Bugs[24]. The support from Find Security Bugs ensures consistency in the testing across the issue types. Additionally, choosing these issues ensures compatibility with CWE identification, as is the case with all issues in Find Security Bugs. None of these 14 CWEs have been covered by the related works [4], [9], and [10].

The test suite's code is artificial, meaning it is written solely for testing purposes and does not constitute part of any actual software. The suite is divided into folders, each representing a specific CWE. The dataset is organized so that one JAR is created for each subfolder. Often, one CWE relates to one subfolder; however, for some CWEs with numerous test cases, the CWE folders are further divided into subfolders. In these cases, a separate JAR is created for each subfolder.

Almost all test cases contain good examples and bad variations, which is useful for benchmarking how well LLMAS can detect true issues and ensuring the efficiency of ignoring false positives. Furthermore, a test case may contain unintentional issues in addition to its intended target type. These flaws should be ignored since they are not part of the test suite and are not present in the ground truth for the dataset. The ground truth for each test case can be found through the source files, where all true positives are contained in methods or classes with "bad" in the name. All issues found that are of the same type as the target case but are not present in these bad methods or classes are considered false positives.

Here follows an example of a test case relating to CWE-643. The file name is `CWE643_Xpath_Injection_connect_tcp_01.java` and contains one flawed method `bad()` with an issue, and one method `good()` that, in turn, calls two methods that contain no issues. All issues of the type CWE-643 found by a tool in the bad method are to be considered true positives, and all other flaws of this type found are to be considered false positives. However, if no issue is found within the flawed method, it is considered a false negative. All other issues found in this file that are not of the type XPath Injection are not to be accounted for in the benchmarking since they are not explicitly a part of the test suite [27].

SpotBugs and LLMAS were applied to all test cases related to the appropriate CWEs to conduct a comprehensive assessment. Scripting was employed to gather the ground truth for each file and label all the bugs from SpotBugs and LLMAS. The detection outcomes were then used to compute evaluation metrics for both tools. All the files analyzed, the results, and the scripts are publicly available through a Bitbucket repository[25].

F. Evaluation Metrics

An experiment was conducted to assess the effectiveness of LLMAS, and its performance was compared with that of SpotBugs. The evaluation focuses on the following evaluation metrics:

1) Precision:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Precision measures the proportion of all issues detected by LLMAS that are actual bug instances. Precision answers the question: "Of all the issues flagged by the tool, how many are real bugs?" This metric ensures that the LLM-enhanced integration effectively filters out false positives (noise) while maintaining or improving the tool's bug detection capability. A high precision score indicates that the system can distinguish true bugs from irrelevant issues.

[23] https://samate.nist.gov/SARD/test-suites/111

[24] https://find-sec-bugs.github.io/bugs.htm

[25] https://bitbucket.org/jsvard/juliet-java-1.3-data/src/main/

2) Recall Rate:

$$\text{Recall Rate} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Recall refers to the proportion of actual bug instances detected by LLMAS. It answers, "How many of the real issues present in the code were successfully identified by the tool?" A high recall rate ensures that the LLM integration does not overlook genuine issues reported by SpotBugs. This metric is vital for determining whether the LLM-enhanced tool can match the recall capabilities of traditional static analysis systems.

3) Processing Time: Processing time refers to the additional latency introduced by the LLM integration during the static analysis process. More specifically, it captures the computational overhead due to interactions with the LLM (such as the Google Gemini model) during code analysis. This metric is critical for determining whether the LLM-enhanced tool can operate efficiently within the time constraints of real-world software development.

The combination of precision, recall rate, and processing time as evaluation metrics is well-justified by existing literature, as they directly relate to the effectiveness of static analysis tools in real-world scenarios. Previous research, such as [4], [9], and [10], has utilized these metrics to evaluate the performance of static analysis tools enhanced with machine learning or LLMs, making them suitable for benchmarking LLMAS.

VI. Results

This section presents the experiment's findings comparing LLMAS to SpotBugs based on the evaluation metrics defined in the methodology. The experiment was conducted between May 9, 2025, and May 12, 2025. The results are organized according to the research questions outlined in the purpose of this study.

A. **RQ1:** *How does LLMAS compare to traditional static analysis tools regarding precision and recall rate?*

The comparison between LLMAS and SpotBugs revealed a modest yet meaningful improvement in precision, with no adverse impact on the overall recall rate. As illustrated in Tables II and III, SpotBugs achieved a precision of 0.3236 and a recall rate of 0.4914 across all tested CWEs. In contrast, LLMAS attained a precision of 0.3673 while maintaining the same recall rate of 0.4914. This corresponds to an absolute increase of 0.0438 in precision, representing an approximate 13.5% improvement.

Although it may appear counterintuitive, the identical recall rate for SpotBugs and LLMAS represents the optimal outcome. This is due to the nature of LLMAS's design: it does not attempt to discover additional true positives but instead filters the results generated by SpotBugs to reduce false positives. Consequently, LLMAS can only maintain or reduce the recall achieved by SpotBugs, but it cannot exceed it. Thus, achieving parity in recall while improving precision underscores the effectiveness of LLMAS in enhancing the usability of static analysis results by streamlining manual inspection.

It is also noteworthy that the performance of both tools varies substantially across different CWE categories, with some CWEs exhibiting significantly higher or lower precision than others. This variation suggests that the effectiveness of static analysis tools, and by extension, LLMAS, can be highly dependent on the nature of the specific vulnerability type.

CWE-ID	TP	FN	FP	Precision	Recall
All	4888	5059	10216	0.3236	0.4914
15	399	301	179	0.6903	0.5700
78	444	252	624	0.4157	0.6379
89	2220	1260	3060	0.4205	0.6379
90	444	252	624	0.4157	0.6379
113	580	1604	3493	0.1424	0.2656
259	51	135	5	0.9107	0.2742
319	259	333	369	0.4124	0.4375
321	17	45	0	1.0000	0.2742
327	34	0	81	0.2957	1.0000
329	16	1	0	1.0000	0.9412
539	3	14	14	0.1765	0.1765
601	333	189	135	0.7115	0.6379
614	16	1	0	1.0000	0.9412
643	72	672	1632	0.0423	0.0968

TABLE II
SpotBugs detection performance for all CWEs tested

CWE-ID	TP	FN	FP	Precision	Recall
All	4888	5059	8419	0.3673	0.4914
15	399	301	160	0.7138	0.5700
78	444	252	460	0.4912	0.6379
89	2220	1260	3020	0.4237	0.6379
90	444	252	479	0.4810	0.6379
113	580	1604	2487	0.1891	0.2656
259	51	135	5	0.9107	0.2742
319	259	333	363	0.4164	0.4375
321	17	45	0	1.0000	0.2742
327	34	0	5	0.8718	1.0000
329	16	1	0	1.0000	0.9412
539	3	14	14	0.1765	0.1765
601	333	189	118	0.7384	0.6379
614	16	1	0	1.0000	0.9412
643	72	672	1308	0.0522	0.0968

TABLE III
LLMAS detection performance for all CWEs tested

The differences in overall precision and recall rate and how they vary across the different CWE categories are further illustrated in Figures 2 and 3.

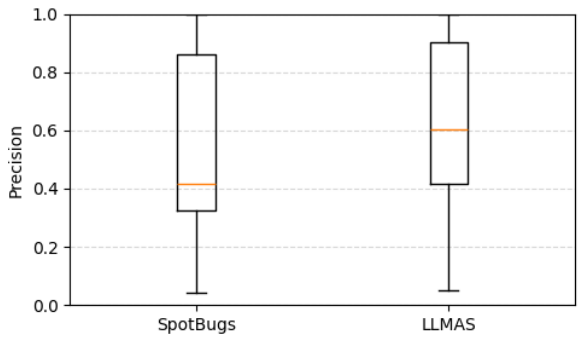


Fig. 2. Precision for All CWEs tested

Figure 2 demonstrates a clear shift in the distribution of precision values between SpotBugs and LLMAS. While both tools exhibit a wide range of precision scores across CWEs, LLMAS shows a notably higher median precision and a narrower interquartile range skewed toward higher values. This indicates that LLMAS not only improves overall precision but does so more consistently across different types of vulnerabilities.

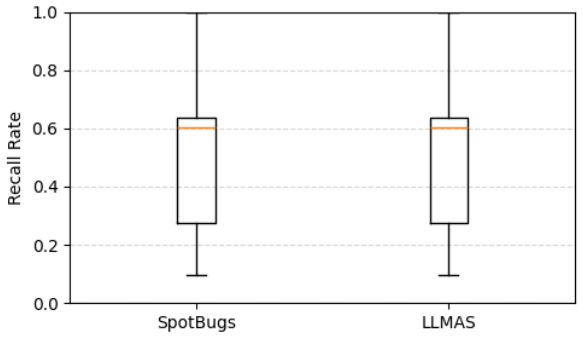


Fig. 3. Recall Rate for All CWEs tested

In contrast, Figure 3 confirms that recall rates remain very stable between the two tools. Both SpotBugs and LLMAS display equal distributions, with identical medians and spreads, reinforcing the earlier observation that LLMAS preserves the recall performance of SpotBugs while filtering out a significant number of false positives.

Taken together, these visualizations emphasize that the improvement in LLMAS precision is not confined to a narrow subset of CWEs but rather represents a general trend across the board. However, the range and distribution also highlight that some CWEs remain more challenging than others for both tools, a nuance that is examined in greater detail in the following subsections.

1) Injection Attacks: Injection attacks represent a critical class of software vulnerabilities due to their potential to allow unauthorized command execution, data exfiltration, or configuration manipulation. In this subsection, the following CWE categories were examined:

- **CWE-15:** External Control of System or Configuration Setting
- **CWE-78:** Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
- **CWE-90:** Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')
- **CWE-113:** Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Response Splitting')

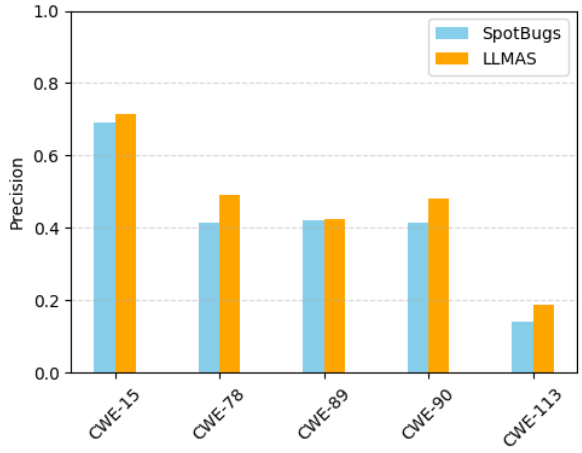


Fig. 4. Precision per CWE for Injection Attacks

As depicted in Figure 4, the precision of both SpotBugs and LLMAS for injection attacks varies across the examined CWEs. For CWE-15, LLMAS exhibits marginally higher precision (0.7138) compared to SpotBugs (0.6903), indicating a slightly improved capability in minimizing false positives related to external configuration control. In the context of CWE-78, LLMAS again surpasses SpotBugs, with a precision score of 0.4912 versus 0.4157. This suggests that LLMAS is more adept at correctly identifying vulnerabilities in this category without over-flagging. For CWE-89, the tools demonstrate nearly identical performance, with precision values around 0.42. This suggests a comparable level of effectiveness, but also highlights a potential ceiling in detection accuracy for both tools in this domain. Concerning CWE-90, LLMAS achieves a more pronounced advantage, registering a precision of 0.4810 compared to SpotBugs' 0.4157. This further reinforces the observation that LLMAS tends to produce fewer false positives across several injection attack types. Finally, both tools perform least effectively in detecting CWE-113, with notably low precision scores. However, LLMAS (0.1891) still outperforms SpotBugs (0.1424), albeit by a narrow margin. This result indicates that both tools face significant challenges in accurately detecting this form of injection attack, possibly due to the complex and context-dependent nature of CRLF-based vulnerabilities.

2) Cryptographic Issues: Cryptographic issues represent a critical class of vulnerabilities due to their potential to

expose sensitive data through weak encryption or poor key management. In this subsection, the following CWE categories were examined:

- **CWE-259:** Use of Hard-coded Password
- **CWE-319:** Cleartext Transmission of Sensitive Information
- **CWE-321:** Use of Hard-coded Cryptographic Key
- **CWE-327:** Use of a Broken or Risky Cryptographic Algorithm
- **CWE-329:** Not Using a Random IV with CBC Mode

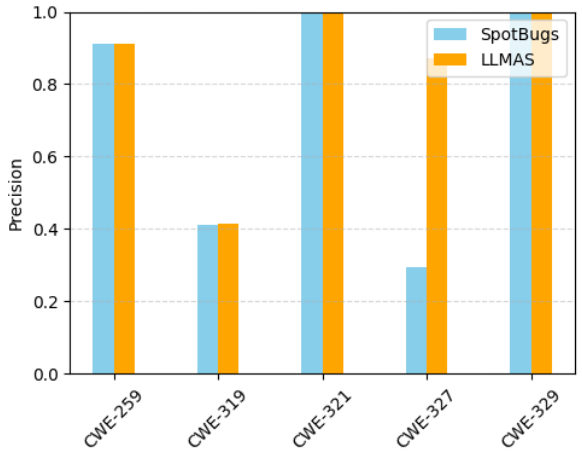


Fig. 5. Precision per CWE for Cryptographic Issues

As depicted in Figure 5, the precision of SpotBugs and LLMAS for cryptographic issues varies considerably across the examined CWEs. For CWE-259, both tools demonstrate exceptionally high and identical precision scores (0.9107), indicating strong capabilities in identifying hard-coded credentials. This high precision may also reflect the simplicity and recognizability of such patterns, although the low number of test cases suggests caution in generalizing these results. In the case of CWE-319, both SpotBugs and LLMAS yield moderate precision scores (0.4124 and 0.4164, respectively). These comparable results imply that neither tool excels at minimizing false positives in this domain, underscoring the complexity of reliably detecting insecure data transmission practices. For CWE-321, both SpotBugs and LLMAS achieve perfect precision (1.0). While this ostensibly indicates flawless detection performance with no false positives, the extremely low number of test cases for this category limits the reliability of this finding. The results should therefore be interpreted with caution. A notable divergence in performance appears in CWE-327. LLMAS attains a significantly higher precision (0.8718) compared to SpotBugs (0.2957), suggesting that LLMAS is more effective in discerning genuinely vulnerable uses of deprecated or insecure algorithms. This contrast points to a higher false positive rate for SpotBugs in this category. However, the low number of test cases again limits the statistical significance of the comparison. Finally, for CWE-329, both tools again report perfect precision (1.0), indicating highly accurate detection of this misconfiguration. Nevertheless, similar to CWE-321, the small number of test cases raises concerns about taking these values at face value.

3) Session and Config Management: Session and configuration management issues represent a critical class of vulnerabilities due to their potential to enable unauthorized access or data leakage through insecure settings or session handling. In this subsection, the following CWE categories were examined:

- **CWE-539:** Information Exposure Through Persistent Cookies
- **CWE-601:** URL Redirection to Untrusted Site ('Open Redirect')
- **CWE-614:** Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
- **CWE-643:** Improper Neutralization of Data within XPath Expressions ('XPath Injection')

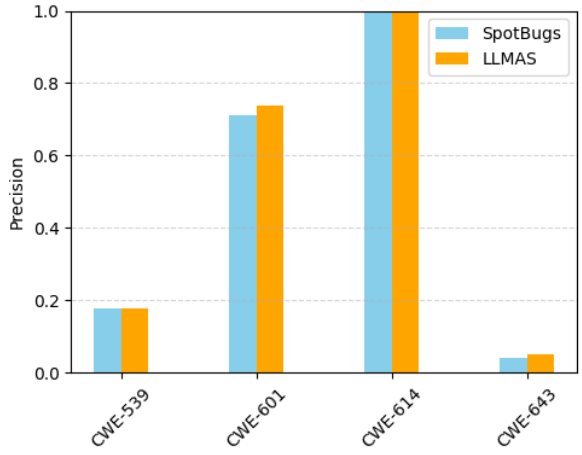


Fig. 6. Precision per CWE for Session and Config Management

As depicted in Figure 6, the precision of SpotBugs and LLMAS for session and configuration management issues varies greatly across the examined CWE categories. For CWE-539, both SpotBugs and LLMAS report a relatively low precision of 0.1765. This suggests that both tools are limited in their ability to accurately detect this type of vulnerability, possibly resulting in a high rate of false positives or a failure to recognize subtle instances. However, the limited number of test cases in this category limits the reliability of this assessment. In contrast, performance improves for CWE-601, where SpotBugs and LLMAS attain precision scores of 0.7115 and 0.7384, respectively. These results indicate a moderate to high level of effectiveness in identifying open redirect issues, with LLMAS showing a slight edge in minimizing false positives. For CWE-614, both tools achieve perfect precision (1.0). This implies that when such misconfigurations are present, both tools are capable of detecting them with complete accuracy. Nevertheless, the very low number of test cases suggests caution in interpreting this result as indicative of broader detection reliability. Finally, CWE-643 presents a considerable

challenge for both tools. SpotBugs achieves a very low precision of 0.0423, while LLMAS fares only marginally better with 0.0522. These figures indicate a significant deficiency in accurately detecting XPath injection vulnerabilities, suggesting a high rate of false positives or missed detections. The nature of XPath-based attacks, which often depend on complex input sanitation and query construction, may partly explain this shortcoming.

B. **RQ2:** *How much additional processing latency does using LLMs for static code analysis introduce?*

The comparison between LLMAS and SpotBugs revealed a significant disparity in processing times. The execution of static code analysis across the full dataset required a total of 434,227 milliseconds (approximately 7.2 minutes) when using SpotBugs. In comparison, LLMAS required 17,493,264 milliseconds (approximately 4.86 hours) to complete the same analysis, indicating a substantial increase in overall processing time. On average, this corresponds to an additional latency of approximately 2,760 milliseconds per file.

CWE-ID	Spotbugs (ms)	LLMAS (ms)	No. Files
All	434227	17064144	6181
15	13290	591307	411
78	22915	820929	456
89	252979	9235509	2280
90	35699	964223	456
113	41414	2960562	1352
259	5671	63588	56
319	17770	573876	260
321	4035	17809	17
327	3510	36854	34
329	3421	16362	16
539	3212	17479	17
601	9169	424616	342
614	3249	16626	16
643	17893	1324404	468

TABLE IV
SPOTBUGS AND LLMAS EXECUTION TIME FOR ALL CWES TESTED

Table IV details the execution times for each CWE category. Across the board, LLMAS incurs significantly higher runtimes. The most extreme case is CWE-89, where LLMAS required 9,235,509 milliseconds (approximately 2.56 hours) to analyze 2,280 files, compared to just 252,979 milliseconds with SpotBugs. Similarly, CWE-113 and CWE-643 show considerable increases, with LLMAS runtimes of 2,960,562 and 1,324,404 milliseconds, respectively. Notably, even CWEs with relatively few files, such as CWE-259, CWE-327, and CWE-539, experienced relatively significant latency increases, highlighting the consistency of the overhead regardless of dataset size.

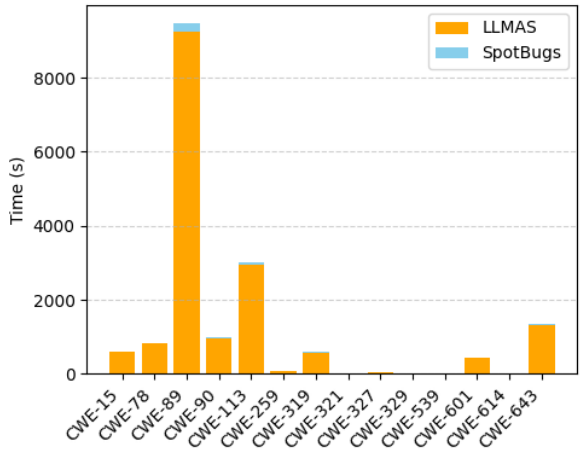


Fig. 7. Stacked Time Breakdown per CWE

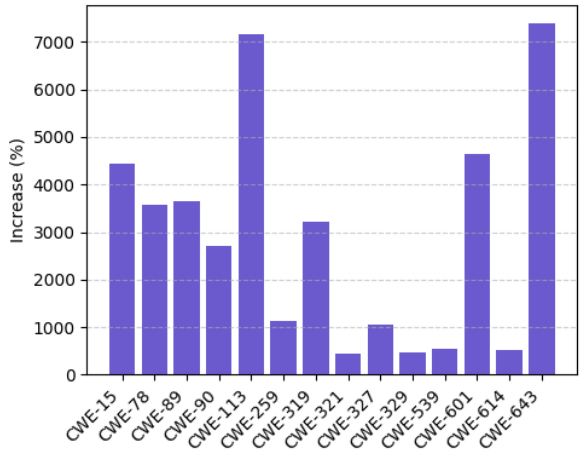


Fig. 8. Average Time Increase in Percentage per CWE

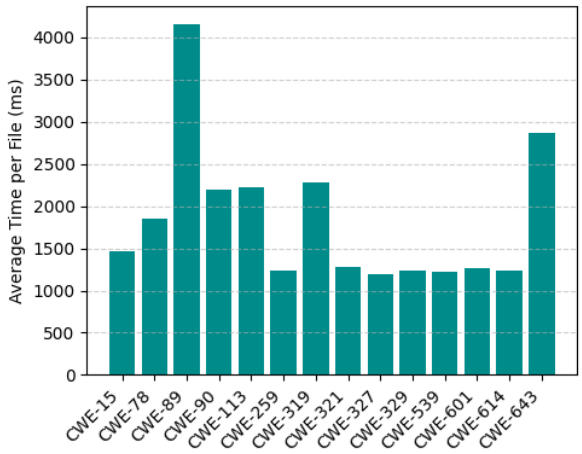


Fig. 9. Average Time per File per CWE

Figure 7 illustrates the total execution time for each CWE, providing a visual comparison between SpotBugs and LLMAS. The figure reinforces the disproportionate processing requirements introduced by LLMAS, particularly for CWEs such as CWE-89, CWE-113, and CWE-643.

To further quantify the impact of the LLM augmentation, Figure 8 shows the percentage increase in processing time introduced by LLMAS relative to SpotBugs for each CWE. The most significant increases are observed for CWE-113 and CWE-643, both exceeding 7000%, followed by CWE-15, CWE-78, CWE-89, CWE-90, and CWE-601, which show increases ranging from approximately 3000% to 4500%. Even for CWEs with lower overall processing times, such as CWE-321, CWE-327, CWE-329, CWE-539, and CWE-614, the percentage increase is still substantial, ranging from around 500% to 1000%. This emphasizes that while the absolute time might be small for certain CWEs, the relative overhead introduced by LLMAS is consistently high.

Figure 9 further breaks down the processing latency by showing the average time per file for each CWE. It can be observed that CWE-89 has the highest average time per file, exceeding 4,000 milliseconds, followed by CWE-643 and CWE-319, which are approximately 2,500 milliseconds and 2,300 milliseconds, respectively. CWE-90 and CWE-113 also show relatively high average times per file, both above 2000 milliseconds. For the remaining CWEs, the average time per file generally falls between 1200 milliseconds and 1800 milliseconds, indicating that even for less complex vulnerabilities, the LLM-based analysis adds a considerable per-file overhead.

The varied averages in processing time for LLMAS can be attributed to several factors, primarily related to the complexity inherent in different CWEs and how these complexities interact with the LLM's capabilities. More complex CWEs, such as CWE-89 and CWE-643, often require intricate data flow analysis, cross-file dependencies, and a nuanced understanding of code semantics. This increased complexity demands more sophisticated reasoning and longer processing times from the LLM. Furthermore, certain CWEs might require the LLM to access and process a larger context window of code or generate more extensive and detailed explanations for detected vulnerabilities, thereby increasing computational load and contributing to the higher average times. The LLM's performance can also be influenced by the diversity and volume of training data for specific CWEs, potentially leading to varied efficiencies in identifying and analyzing different types of vulnerabilities.

VII. Discussion

In this section, we will discuss the results in the context of the related works. Furthermore, limitations and threats to validity will be reasoned about, as well as societal and ethical aspects surrounding this thesis.

A. Comparison to Related Works

Our work mainly builds upon the works by Li et al. [4], Kharkar et al. [9], and Wen et al. [10]. These papers investigate the possibility of reducing false positives by using LLMs to filter the results of static analysis tools. The papers suggest further research in detecting bug types other than those examined in their work and utilizing other static analysis tools; this was the aim of our work. Just as their implementations differ, so does ours in that LLMAS utilizes SpotBugs and Google Gemini. These papers utilize UBITect, Infer, Cppcheck, and CSA as their static analysis tools, and various LLMs are tested.

LLMAS offers a more straightforward implementation, compared to these related works, that enables analysis of all issues supported by SpotBugs and Find Security Bugs. The prompting data in LLMAS consists of the Java file and the bug instances. This is in contrast to the other mentioned works, as their tools only provide code to the LLM that it finds necessary for analysis. Li et al. [4] provides only the function reported in the bug instance, and provides more code on the request of the LLM. Since their tool only detects use before initialization bugs, no bug instances are required, as in LLMAS. Kharkar et al. [9] provide only a half-finished function. This is enough for their approach, which is to let the LLM complete the function and verify if a bug is truly present based on how it is completed. Wen et al. [10] algorithmically extract all the code snippets that the LLM is likely to need and provide them to the LLM for evaluation. Along with this, formatted bug instances are also provided from three different static analysis tools, similar to how LLMAS provides bug instances.

These other works also use several prompting strategies to improve the effectiveness of the LLM. Li et al. [4] perform several steps in prompting the LLM. They task the LLM to specify the initialization and post constraints, which is information specific to use before initialization bugs. They also include examples with the prompts to make the LLM more effective in finding bugs, a technique known as few-shot prompting. Finally, the LLM checks its work throughout the process to increase effectiveness. Wen et al. [10] also provide examples in their prompts; along with this, they utilize chain of thought thinking in the LLM, meaning that they prompt the LLM in a way so it performs thinking step-by-step. This technique aims to improve its effectiveness. Additionally, they perform some post-processing to determine the confidence of the LLM responses and only report the most certain responses. LLMAS uses none of these techniques to improve LLM performance.

As for the LLM parameters, in LLMAS, Gemini's temperature is set to 0 to ensure deterministic results; otherwise, the default values are used. Li et al. [4] utilize GPT-4 and set the temperature to 1, Wen et al. [10] also use GPT and set their temperature to 0.7. They do this because it serves as a good middle ground, in their opinion. They desire more deterministic results but argue that too low a temperature can produce overly simplistic responses and be less effective. Kharkar et al. [9] does not specify the temperature used; an assumption could be made that it is set to 1, the default for the LLM used, GPT-C.

The results for LLMAS show some competency in filtering false positives compared to SpotBugs, which reports 10,216 false positives; LLMAS reports 8,419. With a true positive

amount of 4,888 for LLMAS, this yields a precision of 0.3673, compared to SpotBugs' 0.3236, which also has the same number of true positives. These numbers are still lower than those in the related works on which we based our thesis. LLift created by Li et al. [4] has a precision of 0.5, Kharkar et al. [9] tool has a precision of either 0.606 or 0.854 depending on the bug, and LLM4SA by Wen et al. [10] shows a precision of 0.8113. LLMAS precision is thus significantly lower than that of these other works. The reason for this is hard to pinpoint, although it can be reasoned that it is due to LLMAS's more straightforward integration with the LLM compared to these other works, which use several techniques to improve the tool's effectiveness. The tools of Li et al. [4] and Kharkar et al. [9] also target only certain issues, which allows them to alter the prompts to specifically address these, which could increase precision. LLMAS, on the other hand, is compatible with all issues supported by SpotBugs and Find Security Bugs.

Another reason for these differences in precision could be that our experiments examine CWEs different from those in these related works. Our results show that LLMAS performance differs between issue types. For example, in CWE-327, we observed the most significant improvement, with a precision increase from SpotBugs from 0.2967 to 0.8718 in LLMAS. Other CWEs, such as 259 and 539, show no difference between SpotBugs and LLMAS. This could be because the LLM has a better understanding of some issues than others, especially those that are more common. Kharkar et al. [9] also highlight this phenomenon in their research between resource leak and null dereference, where the precision is raised by 0.248, and the recall rate increases by 0.192.

In terms of recall rate, LLMAS is heavily dependent on SpotBugs. SpotBugs has a recall rate of 0.4914, which is also the recall rate of LLMAS. This is the best-case scenario for LLMAS, as it cannot detect new bugs, due to the prompt design, and thus cannot increase its recall rate. However, poor filtering by LLMAS, such as filtering out true positives, would result in a lower recall rate, which is undesirable. Therefore, compared to SpotBugs, LLMAS shows a recall rate of 1. As stated, the recall rate depends on SpotBugs, and it misses many true positives in the dataset. This is undesirable, as we would rather have too many false positives than miss bugs, since our goal is to prune them. To address this, we ran SpotBugs with high effort and the relaxed flag to capture as many bugs as possible, whether they were true positives or false positives. However, SpotBugs still struggles to detect most bugs.

Compared to the other related works, they also show a higher recall rate than LLMAS; Li et al. [4] 1 (compared to UBITect), Kharkar et al. [9] 0.645 and 0.837, and Wen et al. [10] 0.9464. The reason for these differences in recall rate is probably due to the static analysis tools utilized. As mentioned, LLMAS does not filter out any false positives in our experiment, but still shows a lower recall rate than these works. Li et al. [4] only show the recall rate compared to their static analysis tool, UBITect, which evaluates to 1; this recall rate can be misleading since the recall rate of LLMAS is also 1 compared to SpotBugs. Wen et al. [10] combine bug instances from reports of several static analysis tools, which should increase the recall rate since more bugs are likely to be discovered. Kharkar et al. [9] utilize Infer, which seems to locate more bugs in their experiments than SpotBugs does in ours.

B. Performance overhead

The performance overhead introduced by LLMAS is very significant. While SpotBugs took 7.2 minutes for all files, pruning added another 4.74 hours in our experiment. The comparison of the two should not be overstated, as SpotBugs is machine-dependent, whereas pruning is not. This is because the pruning is performed through an API, and these API calls add additional time, whereas the Python code required to achieve this is very minimal. Still, it highlights the significant time required to filter false positives using LLMs in comparison to traditional static analysis.

The question then is whether the results from LLMAS justify this additional time in the software development process. With the current implementation of LLMAS, we believe that pruning is probably not worth the time. Although LLMAS improves the results of SpotBugs, it is likely not significant enough to justify the additional time required for most software development processes. This effect could be mitigated if LLMAS were integrated into CI/CD workflows, where it could be run automatically, for example, when a commit is pushed to the project repository. The results could then be displayed to the developers without having to manually run LLMAS as a standalone application.

C. Juliet Java 1.3

Our experiment used the Juliet Java 1.3 dataset, which offered a robust dataset that included real issues and false alarms. This was very useful in our testing of LLMAS. However, the dataset required some preparation before it was ready for use in our testing. Many errors were present in the Java files, which stem from the fact that this dataset was written in 2017 and cause problems when compiling the code with a modern JDK. Over 15,000 errors regarding the ambiguous use of an IO class were reported. This error occurs because the helper class IO provided in the dataset was not explicitly imported, and the JDK did not know which IO class to use. To fix this, we had to import the IO helper class in all affected files.

To determine which issues were detected by SpotBugs and LLMAS, a script was devised to collect the ground truth for each Java file. The script traverses each file and collects lines related to bad code. In most cases, these lines are within methods with the word "bad" in their name; in some cases, the whole file is considered bad. This ground truth is then exported to a JSON file, which is subsequently used to perform the bug evaluations. The dataset contains a manifest XML file that lists each code fault and its corresponding line number. However, we collected our ground truth from the dataset because we noticed during testing that SpotBugs, and thus LLMAS, could report the correct faults, but a few lines below or above the

line number present in the manifest file. Thus, the best solution we found was to collect all the bad lines from the dataset and match them with the identified issues. Per the Juliet docs, each reported fault within a bad method or class corresponding to the test case CWEs is to be considered a true positive, and any others of the same type are false positives [27].

D. Limitations and Threats to validity

The use of the Juliet Java 1.3 dataset could be considered a limitation. The data is artificial, created for testing purposes only, and is not part of any real-world software. LLMAS is thus not tested on real-world source code, which could offer more complex code and produce different results. Using a dataset like this was still necessary, as we, the authors, are not experts in software security. Manually labeling issues found in real software would have presented too great a challenge and introduced more significant uncertainty in the results.

Another limitation arises from the CWEs we were able to test in our experiment. We chose all CWEs available in Juliet that were also supported by Find Sec Bugs. Testing more CWEs supported by Find Sec Bugs would have been interesting in the context of our experiment, but it was not possible with the dataset used. We leave these limitations as possible directions for future work. Additionally, the number of test cases for each CWE largely varies. This could skew the results for some of the CWEs that are less represented in Juliet, for example, CWE-327, CWE-329, CWE-539, and CWE-614, all of which contain fewer than 50 related test cases each.

The use of Juliet as a dataset also raises some threats to validity. One concern is that the Java files contain comments that explain the code and highlight where the faults are located. We remove all the comments in LLMAS for our testing to counter this. Another threat to validity is the method and class names, such as ”bad()” or ”good()”, and whether the LLM can interpret these names in its evaluation. From our results, though, we do not think this is the case, since the LLM should filter out false positives far more effectively if that were true. Additionally, LLMs are trained on data from the entire internet, which could also pose a threat to validity. If this dataset has been a part of the LLMs' training, it could potentially detect the issues more effectively than in closed projects that are not publicly available. Again, we do not believe that this has significantly affected the results, as they would likely have been more favorable in that case.

E. Societal and Ethical aspects

Using LLMs in software development raises multiple ethical and social concerns [28]. In terms of static analysis, it raises concerns regarding economics, biases, and loss of expertise. LLMs cost money, especially if you want to access more high-end APIs, and this could be an obstacle for many software developers who cannot afford to incorporate these techniques into their projects. In LLMAS, Gemini Flash Lite was used, which is their lightest version. To this effect, the experiment's cost led to a total cost of 26 kr.

LLMs also pose a threat in that they can have biases; these biases could lead to problems, issues, or flaws remaining unnoticed if they are less frequently discussed or written about online and thus are not as well-known by the LLM. Another threat is the loss of expertise, as human developers become overly reliant on the use of LLMs, thereby decreasing their skills over time. Furthermore, LLMs are trained on data, much of which is publicly available online. These data are often collected without the author's permission, often aiming to reproduce or imitate that work. This could threaten creators of similar work, as the LLM could produce content similar to that of the original, potentially reducing the need for human work in that field.

VIII. Conclusions

In this thesis, we presented LLMAS, which integrates the static analysis tool SpotBugs with the LLM Google Gemini, to filter false positives from SpotBugs' reports. We tested LLMAS on 14 CWEs, comprising 10,056 test cases from the Juliet Java 1.3 dataset, and compared it with standalone SpotBugs. We found that LLMAS filters out some false positives from the SpotBugs reports while not pruning any true positives. LLMAS shows a precision of 0.3673 and a recall rate of 0.4914 compared to SpotBugs, which shows a precision of 0.3236 and the same recall rate. The increase in performance for LLMAS differs between CWEs, where it performs much better on some, e.g., CWE-327, barely better on others, e.g., CWE-643, and some remaining unchanged, e.g., CWE-259.

The main drawback of LLMAS is the additional time required for pruning using the Gemini API. At the current implementation, this overhead is probably considered too large to be used in most software development processes, given the results. Compared to similar tools presented in related works, LLMAS performs worse in terms of precision and recall rate. The lower precision is likely due to the more straightforward LLM integration compared to these other tools. Furthermore, the lower recall rate could be due to the metric's heavy reliance on the static analysis tool used, where SpotBugs seems to struggle in locating many bugs.

For future work, we suggest expanding on this approach to integrate LLMs with static analysis tools for false positive pruning, thereby further enhancing its effectiveness while maintaining the same recall rate as the static analysis tool used. Other static analysis tools and LLMs could be tested to achieve this. The effectiveness of other LLMs could be particularly interesting since LLMAS utilizes the lightest version of Google Gemini. Additionally, testing other prompts and experimenting with temperature and other LLM parameters may also improve the results. LLMAS or a similar tool could also be tested on other CWEs and data, such as real-world projects, to assess their performance on more complex codebases, unlike the artificial code found in the Juliet dataset.

References

[1] T. Drake, "Measuring software quality: a case study," *Computer*, vol. 29, no. 11, pp. 78–87, 1996. [Online]. Available: https://doi.org/10.1109/2.544241

[2] R. Wilhelm, "Software quality assurance by static program analysis," in *Software Quality. Model-Based Approaches for Advanced Software and Systems Engineering: 6th International Conference, SWQD 2014, Vienna, Austria, January 14-16, 2014. Proceedings 6*. Springer, 2014, pp. 1–11. [Online]. Available: https://doi.org/10.1007/978-3-319-03602-1_1

[3] Google DeepMind, "Gemini: A family of multimodal models," 2023 (accessed April 24, 2025). [Online]. Available: https://deepmind.google/technologies/gemini/

[4] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An llm-integrated approach," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 474–499, 2024. [Online]. Available: https://doi.org/10.1145/3649828

[5] Z. Li, S. Dutta, and M. Naik, "Llm-assisted static analysis for detecting security vulnerabilities," *arXiv preprint arXiv:2405.17238*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2405.17238

[6] G. Fan, X. Xie, X. Zheng, Y. Liang, and P. Di, "Static code analysis in the ai era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents," *arXiv preprint arXiv:2310.08837*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2310.08837

[7] W. Ma, S. Liu, Z. Lin, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, L. Li, and Y. Liu, "Lms: Understanding code syntax and semantics for code analysis," *arXiv preprint arXiv:2305.12138*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2305.12138

[8] J. Wang, T. Ni, W.-B. Lee, and Q. Zhao, "A contemporary survey of large language model assisted program analysis," *arXiv preprint arXiv:2502.18474*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.18474

[9] A. Kharkar, R. Z. Moghaddam, M. Jin, X. Liu, X. Shi, C. Clement, and N. Sundaresan, "Learning to reduce false positives in analytic bug detectors," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1307–1316. [Online]. Available: https://doi.org/10.1145/3510003.3510153

[10] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, and T. Cong, "Automatically inspecting thousands of static bug warnings with large language model: How far are we?" *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 7, pp. 1–34, 2024. [Online]. Available: https://doi.org/10.1145/3653718

[11] L. Lavazza *et al.*, "Using static analysis and static measurement for industrial software quality evaluation," *INTERNATIONAL JOURNAL ON ADVANCES IN SOFTWARE*, vol. 14, no. 1-2, pp. 79–87, 2021. [Online]. Available: https://hdl.handle.net/11383/2143991

[12] IEEE Computer Society, "What is software security and why is it important?" 2025 (accessed April 24, 2025). [Online]. Available: https://www.computer.org/resources/software-security/

[13] The Mitre Corporation, "New to cwe," 2025 (accessed April 24, 2025). [Online]. Available: https://cwe.mitre.org/about/new_to_cwe.html

[14] Wikipedia contributors, "Program analysis — Wikipedia, the free encyclopedia," 2025 (accessed April 24, 2025). [Online]. Available: https://en.wikipedia.org/w/index.php?title=Program_analysis&oldid=1269565085

[15] OWASP Foundation, "Source code analysis tools," 2025 (accessed April 24, 2025). [Online]. Available: https://owasp.org/www-community/Source_Code_Analysis_Tools

[16] SpotBugs Team, "Spotbugs webpage," 2025 (accessed April 24, 2025). [Online]. Available: https://spotbugs.github.io/#using-spotbugs

[17] D. H. Hagos, R. Battle, and D. B. Rawat, "Recent advances in generative ai and large language models: Current status, challenges, and perspectives," *IEEE Transactions on Artificial Intelligence*, 2024. [Online]. Available: https://doi.org/10.1109/TAI.2024.3444742

[18] IBM, "What are large language models (llms)?" 2023 (accessed April 24, 2025). [Online]. Available: https://www.ibm.com/think/topics/large-language-models

[19] J. Manyika and S. Hsiao, "An overview of the gemini app," 2025 (accessed April 24, 2025). [Online]. Available: https://gemini.google/overview/?hl=sv

[20] Google, "Thresholds and the confusion matrix," 2025 (accessed April 24, 2025). [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/thresholding

[21] ——, "Classification: Accuracy, recall, precision, and related metrics," 2025 (accessed April 24, 2025). [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

[22] A. Kaur and R. Nayyar, "A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code," *Procedia Computer Science*, vol. 171, pp. 2023–2029, 2020, third International Conference on Computing and Network Communications (CoCoNet'19). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050920312023

[23] L. Flynn and W. Klieber, "Using llms to automate static-analysis adjudication and rationales," *CrossTalk: The Journal of Defense Software Engineering*, 2024. [Online]. Available: https://insights.sei.cmu.edu/library/using-llms-to-automate-static-analysis-adjudication-and-rationales/

[24] Google, "Gemini api documentation," 2025 (accessed May 13, 2025). [Online]. Available: https://ai.google.dev/gemini-api/docs

[25] National Institute of Standards and Technology, "Sard test suites," 2025 (accessed March 27, 2025). [Online]. Available: https://samate.nist.gov/SARD/

[26] P. E. Black, *Juliet 1.3 test suite: Changes from 1.2*. US Department of Commerce, National Institute of Standards and Technology, 2018.

[27] National Institute of Standards and Technology, "Juliet documents," 2025 (accessed May 12, 2025). [Online]. Available: https://samate.nist.gov/SARD/documentation#juliet

[28] B. C. Stahl, *Artificial Intelligence for a Better Future*, ser. SpringerBriefs in Research and Innovation Governance. Springer, 2021. [Online]. Available: https://doi.org/10.1007/978-3-030-69978-9_4

Appendix 1: Contributions

All the work for this thesis was done in close collaboration, with regular meetings for discussion. With that said, the main responsibilities for tasks were divided between the two of us. Below are the lists of this distribution.

1) **Johan Svärd**
   - Abstract
   - Purpose and Contributions
   - Background
   - Related Work
   - Discussion
   - Conclusions
   - LLMAS Tool (Data Processing)
   - Juliet Java 1.3 Preparation/Configuration
   - LLMAS Testing on CWEs
2) **Morgan Teräs**
   - Introduction
   - Research Methodology
   - Results
   - LLMAS Tool (Framework and GUI)
   - LLMAS Testing on CWEs
   - Data Analysis
   - Data Visualization

Appendix 2: Time Plan

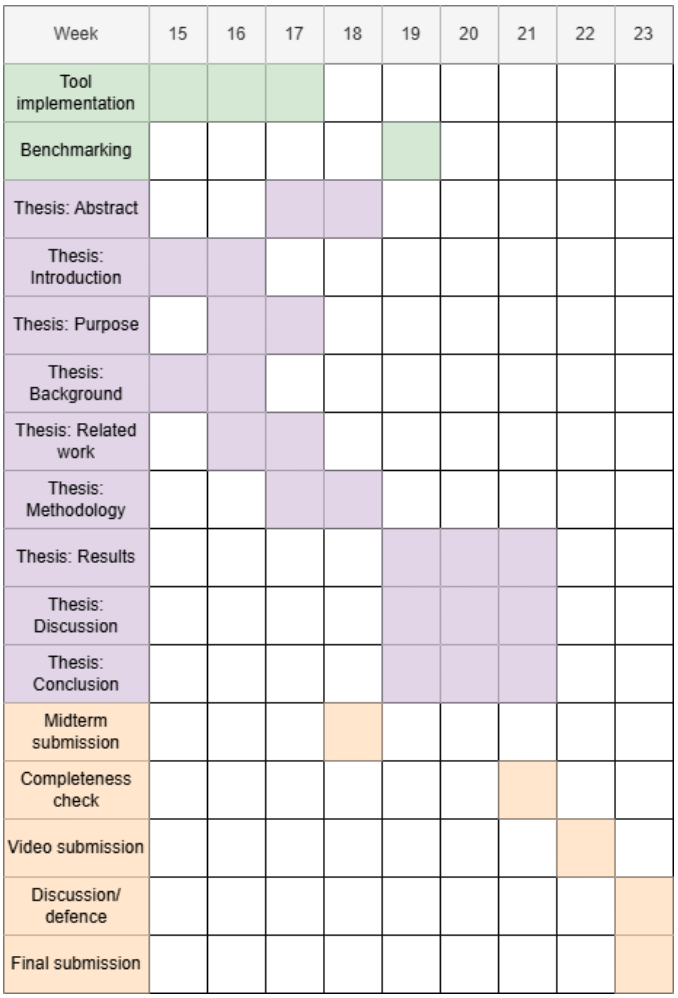


Fig. 10. Initial Time Plan

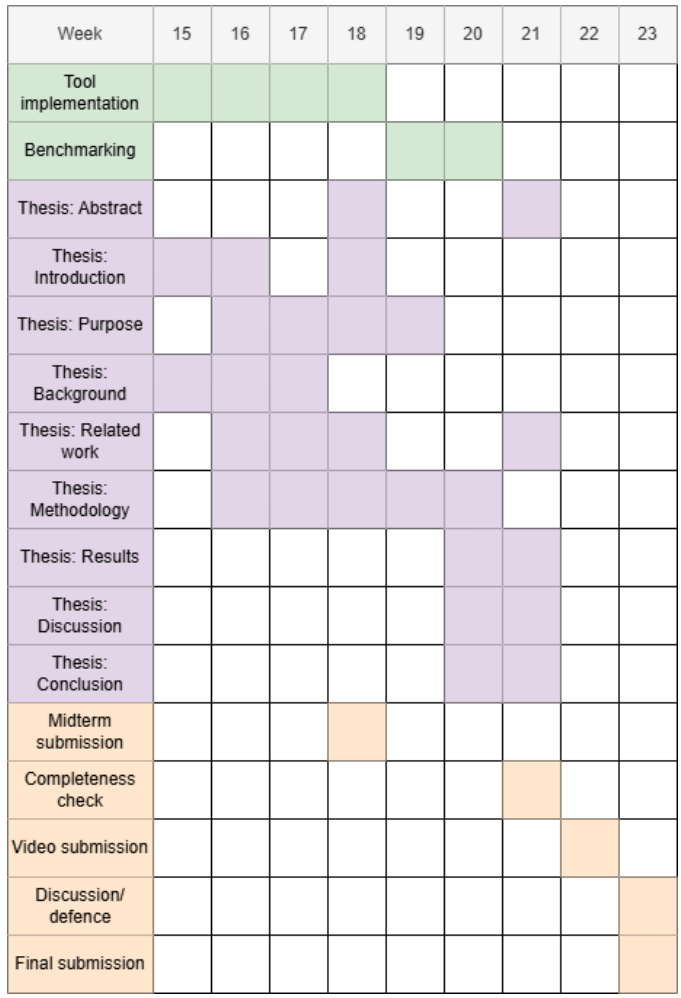
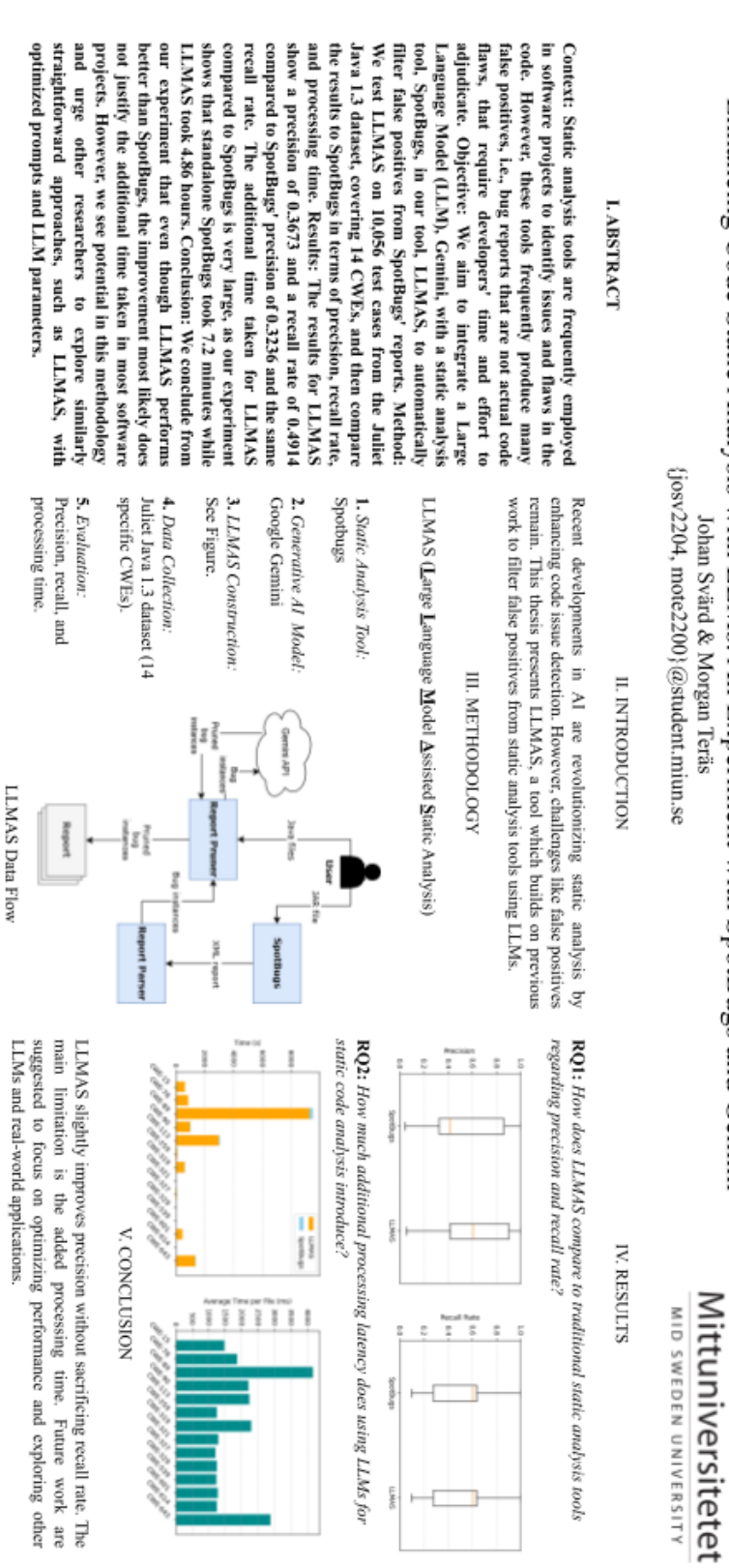


Fig. 11. Actual Time Plan



Enhancing Code Static Analysis with LLMs: An Experiment with SpotBugs and Gemini

Johan Svärd & Morgan Teräs
{josv2204, mote2200}@student.miun.se

Mittuniversitetet
MID SWEDEN UNIVERSITY

I. ABSTRACT

Context: Static analysis tools are frequently employed in software projects to identify issues and flaws in the code. However, these tools frequently produce many false positives, i.e., bug reports that are not actual code flaws, that require developers' time and effort to adjudicate. Objective: We aim to integrate a Large Language Model (LLM), Gemini, with a static analysis tool, SpotBugs, in our tool, LLMAS, to automatically filter false positives from SpotBugs' reports. Method: We test LLMAS on 10,056 test cases from the Juliet Java 1.3 dataset, covering 14 CWEs, and then compare the results to SpotBugs in terms of precision, recall rate, and processing time. Results: The results for LLMAS show a precision of 0.3673 and a recall rate of 0.4914 compared to SpotBugs' precision of 0.3236 and the same recall rate. The additional time taken for LLMAS compared to SpotBugs is very large, as our experiment shows that standalone SpotBugs took 7.2 minutes while LLMAS took 4.86 hours. Conclusion: We conclude from our experiment that even though LLMAS performs better than SpotBugs, the improvement most likely does not justify the additional time taken in most software projects. However, we see potential in this methodology and urge other researchers to explore similarly straightforward approaches, such as LLMAS, with optimized prompts and LLM parameters.

II. INTRODUCTION

Recent developments in AI are revolutionizing static analysis by enhancing code issue detection. However, challenges like false positives remain. This thesis presents LLMAS, a tool which builds on previous work to filter false positives from static analysis tools using LLMs.

III. METHODOLOGY

LLMAS (Large Language Model Assisted Static Analysis)

1. *Static Analysis Tool:* Spotbugs
2. *Generative AI Model:* Google Gemini
3. *LLMAS Construction:* See Figure.
4. *Data Collection:* Juliet Java 1.3 dataset (14 specific CWEs).
5. *Evaluation:* Precision, recall, and processing time.

LLMAS Data Flow

IV. RESULTS

RQ1: *How does LLMAS compare to traditional static analysis tools regarding precision and recall rate?*

RQ2: *How much additional processing latency does using LLMs for static code analysis introduce?*

V. CONCLUSION

LLMAS slightly improves precision without sacrificing recall rate. The main limitation is the added processing time. Future work are suggested to focus on optimizing performance and exploring other LLMs and real-world applications.

Fig. 12. Poster